Angular + Django API Documentation

Unified documentation for the Angular frontend & Django API backend

---

Overall Project Review — Angular + Django

This project uses a modern SPA architecture where the **Angular frontend** communicates with a **Django REST API backend**. Angular handles UI rendering, user interaction, authentication state, and routing. Django provides authentication, database management, and REST API endpoints.

- **Angular** renders UI, handles authentication, routing, components, services, guards.
- **Django API** provides secure endpoints for user, posts, comments, likes, downloads.
- **JWT** secures authentication between Angular and Django.
- **Interceptor** injects JWT into every request.
- **Guards** restrict access to protected routes.

📘 Angular Application Documentation

Clean Django-API style explanation for all Angular components, services, guards, and interceptors

---

Complete Documentation Index

Section	Description	Link
Angular Application Documentation		
1. Components	All Angular components with responsibilities & services used	Go
1.1 Nav Component	Handles login, signup, Google login, search	Go
1.2 Home Component	Homepage feed, infinite scroll, top posts/users	Go
1.3 UserProfile Component	User profile, edit, upload post, delete post	Go
1.4 ViewPdfPost Component	Displays PDF, comments, likes, downloads	Go
1.5 Footer Component	Static footer section	Go
2. Services	AuthService & ProfileService Django-style explanations	Go
3. Guards	Angular route protection (LoginGuard)	Go
4. Interceptors	JWT Token Interceptor explanation	Go
5. Authentication Flow	Login, Google Login, Guard Flow, Interceptor Flow	Go
Django API Documentation		
Django API Overview	Project overview, features, backend purpose	Go
Models Documentation	User, Post, Comment models	Go
Serializers Documentation	JWT, User, Post, Comment, Mini serializers	Go
Permissions Documentation	Unified Role-Based Permission System	Go
Views Documentation	All REST API endpoints (Auth, Posts, Comments, Search)	Go
URLs Documentation	API routing table	Go
Settings Documentation	Django REST framework settings, JWT, media config	Go
Django-API Reference Endpoints	All API end points Quick Reference	Go

---

1. COMPONENTS

Each component below has:

- Component Name
- What it is used for (purpose)
- What services it consumes
- How it interacts with the Angular app

🔹 1.1 Nav Component

Purpose:
The navigation header for the entire website. Handles user authentication (login/signup/Google login), search bar, profile menu, and routing.

Main Responsibilities:

- Show login/signup modals
- Manage user session state
- Display user profile picture & dropdown menu
- Execute search queries for users and posts
- Initialize Google OAuth button
- Redirect between pages

Services used:

1) Auth Service

- login(username, password) → logs user in
- register(payload) → creates a new user
- googleBackendLogin(token) → handles Google login response
- Maintains **user$** observable to broadcast logged-in user info

Explanation:
The Nav component subscribes to **auth.user$**. This means the Nav bar updates instantly whenever login/signup happens. It also calls Auth service for login/signup form submissions.

2) Profile Service

- search(query) → returns { users, posts }
- Provides small user-info snippets for the UI

Explanation:
When a user types into the search bar, the Nav component calls the ProfileService to retrieve matching results. It shows them in a results dropdown.

🔹 1.2 Home Component

Purpose:
This is the landing page of the app. Shows the homepage feed of posts, infinite scroll, top users, and top posts.

Main Responsibilities:

- Load feed posts (paginated)
- Implement infinite scroll using IntersectionObserver
- Show top users & top liked posts
- Navigate to post view pages

Services used:

1) Profile Service

- getHomePosts(page, limit) → homepage feed
- getTopUsersByLikes() → top creators
- getTopPosts() → trending posts

2) Auth Service: Used simply to identify whether user is logged in (for like buttons, saving, etc.)

🔹 1.3 UserProfile Component

Purpose:
Handles all profile-related operations for the current logged-in user.

Main Responsibilities:

- Display user info
- Edit username, email, bio, DOB
- Upload profile picture
- Show user posts
- Add a new post (PDF upload)
- Delete posts

Services used:

1) Profile Service

- myProfile() → load user data
- updateProfile(formData) → edit profile
- getMyPost() → list user posts
- addPost(formData) → create PDF post
- deleteMyPost(id) → delete a post

Explanation:
UserProfile component delegates all backend operations to ProfileService and only manages UI state.

2) Auth Service: Used to fetch the authenticated user details and token when making secure requests.

🔹 1.4 ViewPdfPost Component

Purpose:
Displays a single PDF post with viewer UI, comments, likes, and download counter.

Main Responsibilities:

- Load a single PDF post
- Sanitize PDF URL using DomSanitizer
- Add comments
- Delete comments
- Like a post
- Increase download counter

Services used:

1) Profile Service

- getSinglePost(id)
- addComment(postId, payload)
- deleteComment(postId, commentId)
- likePost(id)
- downloadPost(id)

2) Auth Service: Ensures the user is logged in before commenting, liking, or downloading.

🔹 1.5 Footer Component

Purpose: Static footer showing year, links, brand.

Services used: None — simple display component.

2. SERVICES

Below is a generic explanation of how your services behave, Django-style.

◆ 2.1 Auth Service

Purpose:
Handles login, logout, signup, and authentication token management.

Key Features:

- Maintains authenticated user state (user$)
- Sends login/register HTTP requests
- Stores JWT token in localStorage
- Sends authenticated requests using the interceptor
- Supports Google OAuth login

Django-style explanation:
Think of AuthService like Django's authenticate() + login() functions combined with token handling. Components call AuthService → AuthService calls backend → updates user state observable.

◆ 2.2 Profile Service

Purpose:
All user-related and post-related backend operations.

Operations Provided:

- Fetch profile, update profile
- Upload images & PDF posts
- Search users/posts
- Fetch top users/top posts
- Post interactions: comment, like, download
- Retrieve homepage feed

Django-style explanation:
This service acts like Django's "views for user and post models" but exposed through Angular. Components never directly handle backend URLs — ProfileService does the mapping.

---

3. GUARDS (Angular Route Protection)

◆ Login Guard

Purpose:
Prevents unauthenticated users from accessing pages like profile, post creation, etc.

How it works:

- Guard checks AuthService for isLoggedIn or token existence.
- If no user is logged in:
  - Redirects to home page or login modal
  - Cancels route activation

Django analogy:
Equivalent to Django's @login_required decorator on a view.

---

4. INTERCEPTORS

◆ Auth Token Interceptor

Purpose:
Automatically attaches JWT token to every HTTP request.

How it works:

- Intercepts outgoing request
- Reads JWT token from localStorage
- Adds Authorization: Bearer <token> header
- Sends updated request to server

Django analogy:
Behaves like Django REST Framework's Authentication middleware, but on the frontend side.

---

5. AUTH FUNCTIONS & FLOW

Login Flow

- User enters username/password
- Nav component calls auth.login()
- Backend returns token & user details
- Token stored in browser
- user$ observable updates → whole app knows user is logged in

Google Login Flow

- Google SDK generates credential token
- Nav sends token → auth.googleBackendLogin()
- Backend validates Google token → returns JWT

Protected Route Flow (Guard)

- User tries to visit profile page
- LoginGuard checks AuthService
- If not logged in → redirect

Interceptor Flow

- Any HTTP request
- Add token header
- Backend authenticates
- Response continues normally

📄 End of Angular App Documentation

📘 Django API Documentation

Welcome to the complete backend documentation for the QuickThrough Django REST API. This index page helps you navigate to documentation for models, serializers, permissions, views, URLs, and settings.

🔷 API Project Overview

This Django REST application powers user authentication, Google login, profile management, posts with PDF preview generation, comments, likes, leaderboard ranking, and pagination-based home feeds. The API is designed to work with Angular frontend clients, providing secure JWT authentication and clean, structured JSON responses.

- ✔️ Custom User Model (role-based)
- ✔️ JWT Authentication + Google OAuth
- ✔️ PDF Upload + Automatic Image Preview
- ✔️ Comments, Likes, Leaderboards
- ✔️ Optimized serializers with absolute media URLs
- ✔️ Unified Role-Based Permission System

📚 Documentation Sections

📌 Models Documentation

Detailed explanation of User, Post, Comment models and upload helper functions.

View Models Doc

📌 Serializers Documentation

All serializers including authentication, user profile, posts, comments, search, and leaderboard.

View Serializers Doc

📌 Permissions Documentation

Unified RBAC permission system controlling access across users, posts, and comments.

View Permissions Doc

📌 Views Documentation

All API endpoints: Auth, Profile, Posts, Comments, Search, Leaderboard, Google Login.

View Views Doc

📌 URLs Documentation

Complete API routing table with explanation of every endpoint.

View URLs Doc

📌 Settings Documentation

Project configuration including JWT, REST framework, CORS, media paths, and custom user model.

View Settings Doc

📘 models.py

1. Imports Used in models.py

```
from django.db import models
from django.contrib.auth.models import AbstractUser
from pdf2image import convert_from_path
from PIL import Image
import os
import fitz
from django.conf import settings
from django.utils import timezone
```

Medium Detailed Explanation

- **models** — provides Django’s ORM features used to define database tables.
- **AbstractUser** — lets you extend Django’s default user model without breaking authentication. This is the recommended way when adding custom fields such as `role` or `image`.
- **convert_from_path** (pdf2image) — an optional PDF-to-image converter. The app primarily uses PyMuPDF (`fitz`), but pdf2image is still imported, possibly for fallback or older code compatibility.
- **PIL.Image** — Python Imaging Library; can manipulate images (resize, crop, optimize). Useful if you plan to compress generated PDF previews.
- **os** — essential for building file paths such as `MEDIA_ROOT/pdf_previews/...jpg`.
- **fitz (PyMuPDF)** — the main library used here for opening PDFs, reading their pages, and exporting them as pixmaps (images). Very fast and reliable.
- **settings** — lets the model know where the MEDIA folder is located when saving preview images.
- **timezone** — ensures timestamps are timezone-aware, which is important for multi-region deployments.

2. Upload Path Helper Functions

```
def pdf_upload_path(instance, filename):
    return f"pdfs/{filename}"

def preview_upload_path(instance, filename):
    return f"pdf_previews/{filename}"

def userimg_upload_path(instance, filename):
    return f"userimage/{filename}"
```

Medium Detailed Explanation

- Each function defines where uploaded files are saved inside the `MEDIA_ROOT` directory.
- **pdf_upload_path** Stores all uploaded PDFs in a clean directory: `media/pdfs/`. This makes file management easier (deletions, migrations, backups).
- **preview_upload_path** All auto-generated JPG previews from PDFs go into: `media/pdf_previews/`. Keeping generated assets separate from uploaded files avoids confusion.
- **userimg_upload_path** Stores profile images inside: `media/userimage/`. The separation helps with caching/CDN optimization.
- **Why not store everything in one folder?** Organizing by type:
    - improves security
    - makes cleanup scripts safer
    - avoids naming conflicts between PDFs and preview images

3. User Model (Custom User)

```
class User(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('staff', 'Staff'),
        ('user', 'User'),
    )

    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='user')
    image = models.ImageField(upload_to=userimg_upload_path, blank=True, null=True)
    bio = models.CharField(max_length=255, blank=True, null=True)
    bdo = models.DateTimeField(blank=True, null=True)
    verified = models.BooleanField(default=False)
    verified_at = models.DateTimeField(null=True, blank=True)
    blocked = models.BooleanField(default=False)
    blocked_at = models.DateTimeField(null=True, blank=True)
    deleted = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        if self.role in ['admin', 'staff']:
            self.verified = True

        if self.verified and self.verified_at is None:
            self.verified_at = timezone.now()

        if self.blocked and self.blocked_at is None:
            self.blocked_at = timezone.now()

        super().save(*args, **kwargs)
```

Medium Detailed Explanation

- **Extends AbstractUser** This is the safest and most flexible way to customize Django’s auth model while keeping login, sessions, and admin panel working normally.
- **role** Defines permission level. Used heavily in `permissions.py`. Role-based access control is cleaner than checking groups for small projects.
- **image** Optional field pointing to the user’s profile photo. Allows display in posts, comments, and headers.
- **verified / verified_at** Indicates whether the account is “trusted.” Automatically marks admin/staff as verified. Useful for hiding spam accounts in public feeds.
- **blocked / blocked_at** Allows moderators to block users without deleting their data. Timestamps allow audit logs (e.g., when was the user blocked?).
- **save() logic** Medium-level reasoning:
    - If user becomes verified → timestamp recorded once.
    - If user is blocked → timestamp recorded once.
    - Admin/staff always remain verified to prevent accidental override.

  This avoids needing database triggers or Django signals.

4. Post Model (PDF + Auto Preview)

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    file = models.FileField(upload_to=pdf_upload_path)
    image = models.ImageField(upload_to=preview_upload_path, blank=True, null=True)
    author = models.ForeignKey(User, on_delete=models.CASCADE, related_name="posts")
    likes = models.ManyToManyField(User, related_name="liked_posts", blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now_add=True)

    def save(self, *args, **kwargs):
        super().save(*args, **kwargs)

        if self.file and not self.image:
            doc = fitz.open(self.file.path)
            page = doc.load_page(0)
            pix = page.get_pixmap()

            img_filename = os.path.splitext(os.path.basename(self.file.path))[0] + ".jpg"
```

```
            img_path = os.path.join(settings.MEDIA_ROOT, "pdf_previews", img_filename)

            os.makedirs(os.path.dirname(img_path), exist_ok=True)
            pix.save(img_path)

            self.image = f"pdf_previews/{img_filename}"
            super().save(update_fields=["image"])
```

Medium Detailed Explanation

- **title, description** — Standard metadata for the post. Description is optional to allow very simple posts.
- **file** — The main uploaded PDF document. FileField automatically manages file uploads and handles file storage.
- **image** — Will store the generated preview thumbnail from page 1 of the PDF.
- **save() override** — The most important part of this model:
    - First `super().save()` saves the model so the PDF file exists on disk.
    - Then the model checks whether there is a PDF but no preview yet.
    - Uses `fitz.open()` to read the PDF efficiently.
    - Extracts the first page and converts it to an image (pixmap).
    - Builds the output filename using the same PDF name but as JPG.
    - Saves the JPG to `media/pdf_previews/`.
    - Updates the `image` field using `update_fields` (faster and avoids re-saving the entire object).
- **likes** — Many-to-many relationship allowing users to like a post. Django automatically handles join tables behind the scenes.
- **created_at / updated_at** — Automatically set timestamps. Helpful for sorting posts in feeds.

5. Comment Model

```
class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE, related_name="comments")
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    parent = models.ForeignKey('self', null=True, blank=True,
                               related_name="replies", on_delete=models.CASCADE)
    text = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now_add=True)
```

Medium Detailed Explanation

- **post** — Connects the comment to its parent post. `related_name="comments"` lets you do: `post.comments.all()`.
- **user** — The user who wrote the comment.
- **parent** — Key field enabling replies: - If parent = null → top-level comment - If parent != null → it's a reply to another comment This structure allows unlimited nesting.
- **text** — The comment's content.
- **created_at / updated_at** — Timestamp metadata. Currently both use `auto_now_add`; if you want update tracking, change `updated_at` to `auto_now=True`.
- **Benefit of this design**: - Supports threaded discussions - Simple queries for both top-level comments and replies - Efficient database relationships

📘 serializers.py

1. Imports Used in serializers.py

```
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework import serializers
from django.contrib.auth import get_user_model
from django.conf import settings
from .models import Post, Comment

User = get_user_model()
```

Medium Detailed Explanation

- **TokenObtainPairSerializer** — Base serializer from SimpleJWT used to generate access/refresh tokens.
- **serializers** — DRF's serializer classes (ModelSerializer, fields, validation).
- **get_user_model()** — Safely loads your custom User model instead of using Django's default.
- **settings** — Used for generating absolute file/image URLs.
- **Post, Comment** — Import models used in post and comment serializers.

2. MyTokenObtainPairSerializer

```
class MyTokenObtainPairSerializer(TokenObtainPairSerializer):

    def validate(self, attrs):
        data = super().validate(attrs)
        data["id"] = self.user.id
        data["username"] = self.user.username
        data["email"] = self.user.email
        data["role"] = self.user.role

        request = self.context.get("request")
        if self.user.image:
            if request:
                data["image"] = request.build_absolute_uri(self.user.image.url)
            else:
                data["image"] = self.user.image.url
        else:
            data["image"] = None
```

```
        return data
```

Medium Detailed Explanation

- This serializer modifies SimpleJWT's login response.
- **super().validate()** Creates access + refresh tokens. We extend the returned dictionary.
- **Adds extra user data**:
  - ID
  - username
  - email
  - role

  This avoids making an extra API call for user profile after login.
- **Image Handling** If the user has a profile image:
  - Use `build_absolute_uri()` so frontend gets a full URL.
  - Fallback: relative path when request is missing.
- Useful for SPAs (Angular, React) since login returns full ready-to-use user info.

3. RegisterSerializer

```
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, min_length=4)

    class Meta:
        model = User
        fields = ["id", "username", "email", "password"]

    def create(self, validated_data):
        user = User(
            username=validated_data["username"],
            email=validated_data["email"]
        )
        user.set_password(validated_data["password"])
        user.save()
        return user
```

Medium Detailed Explanation

- Creates new users with username, email, and password.
- **password = write_only** Ensures the password **never appears in API responses**.
- **set_password()** Hashes the password using Django’s secure hashing system.
- Return value is the newly created User instance.
- JWT tokens are issued later in the Register API view.

4. MeSerializer (User Profile)

```
class MeSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ["id", "username", "email", "bdo", "bio", "image", "verified", "role"]
        read_only_fields = ["id", "verified"]

    def to_representation(self, instance):
        data = super().to_representation(instance)
        request = self.context.get("request")

        if instance.image:
            data["image"] = request.build_absolute_uri(instance.image.url)
        else:
            data["image"] = None

        return data
```

Medium Detailed Explanation

- This serializer is used in the **/me/** profile endpoint.
- **read_only_fields** Prevents editing ID or verified status via this serializer.
- **to_representation()** Converts file path → full absolute URL (useful for frontend images).
- **Why override?** DRF does NOT automatically convert media file fields into full URLs.
- Provides a user-friendly profile response.

5. UserUpdateSerializer

```
class UserUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ["id", "username", "email", "bio", "bdo", "image"]
        read_only_fields = ["id"]
        extra_kwargs = {
            "email": {"required": False},
            "username": {"required": False},
            "bio": {"required": False},
            "bdo": {"required": False},
            "image": {"required": False},
        }

    def validate_email(self, value):
        user = self.context["request"].user
        if User.objects.exclude(id=user.id).filter(email=value).exists():
            raise serializers.ValidationError("Email already exists.")
        return value
```

Medium Detailed Explanation

- Used in update-profile endpoint for PATCH requests.
- **extra_kwargs** Makes all fields optional, allowing partial updates.
- **validate_email()** Prevents the user from updating email to another user's email.
- DRF passes request via serializer context → used here to get the current user.

6. Post & HomePost Serializers

```
class PostSerializer(serializers.ModelSerializer):
    file = serializers.SerializerMethodField()
    image = serializers.SerializerMethodField()
    is_liked = serializers.SerializerMethodField()
    likes_count = serializers.SerializerMethodField()
    comments = serializers.SerializerMethodField()

    class Meta:
        model = Post
        fields = [
            "id", "title", "description", "file", "image",
            "author", "likes", "is_liked", "likes_count",
            "comments", "created_at", "updated_at"
        ]
        read_only_fields = ["author", "likes", "created_at", "updated_at"]

    def get_file(self, obj):
        return settings.MEDIA_URL + obj.file.name if obj.file else None

    def get_is_liked(self, obj):
        user = self.context["request"].user
        return obj.likes.filter(id=user.id).exists() if user.is_authenticated else False

    def get_likes_count(self, obj):
        return obj.likes.count()

    def get_comments(self, obj):
        top = obj.comments.filter(parent=None).order_by("-created_at")
        return CommentMiniSerializer(top, many=True, context=self.context).data

    def get_image(self, obj):
        return settings.MEDIA_URL + obj.image.name if obj.image else None

    def to_representation(self, instance):
        data = super().to_representation(instance)
        request = self.context["request"]

        data["image"] = (
            request.build_absolute_uri(instance.image.url) if instance.image else None
        )
        data["file"] = (
            request.build_absolute_uri(instance.file.url) if instance.file else None
        )
        return data
```

Medium Detailed Explanation

- **SerializerMethodField** Allows dynamic fields: absolute URLs, like-counts, comments list.
- **is_liked** Checks whether the logged-in user has liked the post.
- **comments** Returns ONLY top-level comments (replies handled separately).
- **to_representation()** Final transformation step → ensures URLs are absolute.
- **HomePostSerializer** (not shown fully)
  - Used for public pages or home feed.
  - Includes author mini profile.
  - Shows likes count, preview image, comments summary.

7. Mini Serializers

```
class UserMiniSerializer(serializers.ModelSerializer):
    image = serializers.SerializerMethodField()

    class Meta:
        model = User
        fields = ["id", "username", "image", "verified"]

    def get_image(self, obj):
        request = self.context.get("request")
        return request.build_absolute_uri(obj.image.url) if obj.image else None


class CommentMiniSerializer(serializers.ModelSerializer):
    user = UserMiniSerializer(read_only=True)

    class Meta:
        model = Comment
        fields = ["id", "text", "created_at", "user"]
```

Medium Detailed Explanation

- **UserMiniSerializer** Lightweight version of user details used in comments and homeposts.
- **CommentMiniSerializer** Displays comment text + user info without heavy nested data.
- **SearchUserSerializer / SearchPostSerializer** Used for search API — returns trimmed-down details.
- **TopUserSerializer / TopPostSerializer** Used by leaderboard endpoints to show popularity metrics.

📘 permissions.py

1. Imports Used in permissions.py

```
from rest_framework.permissions import BasePermission, SAFE_METHODS
from django.contrib.auth import get_user_model
from .models import Post, Comment

User = get_user_model()
```

Medium Detailed Explanation

- **BasePermission** — The base class to create custom DRF permissions.
- **SAFE_METHODS** — Contains GET, HEAD, OPTIONS. These do NOT modify data, so many APIs allow them publicly.
- **get_user_model()** — Loads your custom User model with extra fields like role, verified, blocked.
- **Post, Comment** — Needed to apply object-level rules depending on the object type.

2. UnifiedRBACPermission Class

```
class UnifiedRBACPermission(BasePermission):

    def has_permission(self, request, view):
        if request.method in SAFE_METHODS:
            return True

        if not request.user.is_authenticated:
            return False

        return True


    def has_object_permission(self, request, view, obj):

        if request.method in SAFE_METHODS:
            return True

        user = request.user

        if user.role in ['admin', 'staff']:
            return True

        if isinstance(obj, Post):
            return obj.author == user

        if isinstance(obj, Comment):
            return obj.user == user

        if isinstance(obj, User):
            return obj.id == user.id

        return False
```

Medium Detailed Explanation

- **Single permission system for entire project** Instead of writing permissions for every model, everything is handled here.

- **has_permission()** — View-Level Access
  - Allows public access to GET, HEAD, OPTIONS.
  - For POST / PUT / DELETE → user must be logged in.
  - No object-level checks yet; those occur in `has_object_permission`.

- **has_object_permission()** — Object-Level Rules Executed when modifying a specific object (post, comment, or user).

3. Object-Level Permission Rules

A) SAFE METHODS (GET, HEAD, OPTIONS)

```
if request.method in SAFE_METHODS:
    return True
```

- All users (even anonymous) can read posts, comments, or profiles.

B) Admin / Staff Override

```
if user.role in ['admin', 'staff']:
    return True
```

- Admins & staff bypass all restrictions.
- They can edit/delete any post, comment, or user.

C) Object Rules for Post

```
if isinstance(obj, Post):
    return obj.author == user
```

- Users can modify **only posts they authored**.
- Prevents one user from editing another user's content.

D) Object Rules for Comment

```
if isinstance(obj, Comment):
    return obj.user == user
```

- Users can edit/delete only their own comments.
- Admins/staff still override.

E) Object Rules for User Profile

```
if isinstance(obj, User):
    return obj.id == user.id
```

- Users can update only their own profiles.
- Prevents unauthorized profile edits.

F) Fallback Rule

```
return False
```

- If the object type doesn't match Post, Comment, or User → deny.

📘 views.py

1. Imports in views.py

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from rest_framework.parsers import MultiPartParser, FormParser
from rest_framework_simplejwt.views import TokenObtainPairView
from rest_framework.response import Response
from rest_framework import status
from rest_framework_simplejwt.tokens import RefreshToken

from django.shortcuts import get_object_or_404
from django.http import Http404
from django.contrib.auth import get_user_model
from django.db.models import Q, Count

from .models import Post, Comment
from .serializers import (
    MyTokenObtainPairSerializer, RegisterSerializer, MeSerializer,
    UserUpdateSerializer, PostSerializer, CreatePostSerializer,
    HomePostSerializer, CommentMiniSerializer, SearchPostSerializer,
    SearchUserSerializer, TopUserSerializer, TopPostSerializer
)
from .permissions import UnifiedRBACPermission

from google.oauth2 import id_token
from google.auth.transport import requests as grequests
from django.conf import settings
```

Medium Detailed Explanation

- **APIView** — Base class for all custom API endpoints.
- **TokenObtainPairView** — Used for JWT login endpoints.
- **MultiPartParser / FormParser** — Required for uploading files (PDF, images).
- **get_object_or_404** — Safe DB lookups returning 404 automatically.
- **Q** — Used for performing OR queries in search.
- **Count** — Used to calculate likes count in leaderboard.
- **Google OAuth imports** — Verify Google ID tokens.
- **User model** — Loaded via get_user_model() to support custom user fields.

2. GoogleAuthAPIView — Google Login Flow

```
class GoogleAuthAPIView(APIView):
    permission_classes = []  # Public endpoint

    def post(self, request):
        token = request.data.get("id_token")

        if not token:
            return Response({"error": "Missing id_token"}, status=400)

        try:
            google_info = id_token.verify_oauth2_token(
                token,
                grequests.Request(),
                settings.GOOGLE_CLIENT_ID
            )
        except Exception:
            return Response({"error": "Invalid Google token"}, status=400)

        email = google_info.get("email")
        name = google_info.get("name", "")
        picture = google_info.get("picture")

        if not email:
            return Response({"error": "Google email not found"}, status=400)

        # Existing or new user
        user = User.objects.filter(email=email).first()
        created = False

        if not user:
            # Create unique username
            base_username = email.split("@")[0]
            username = base_username
            i = 1
            while User.objects.filter(username=username).exists():
                username = f"{base_username}{i}"
                i += 1

            user = User(username=username, email=email)
            user.set_unusable_password()
            user.save()
            created = True

        if name and not user.first_name:
            user.first_name = name
            user.save()

        image_url = request.build_absolute_uri(user.image.url) if user.image else None

        refresh = RefreshToken.for_user(user)
        return Response({
            "created": created,
            "id": user.id,
            "username": user.username,
            "email": user.email,
            "role": user.role,
            "image": image_url,
            "access": str(refresh.access_token),
            "refresh": str(refresh),
        }, status=200)
```

Medium Detailed Explanation

- Client sends a Google ID token → backend verifies it using Google’s public keys.
- If the email matches an existing user → login.
- If not → a new user is created with a unique username.
- No password is set → account can only be accessed via Google login.
- Issues JWT access + refresh tokens for authentication.

3. LoginView — JWT Login

```
class LoginView(TokenObtainPairView):
    serializer_class = MyTokenObtainPairSerializer
```

Medium Detailed Explanation

- Extends SimpleJWT's login endpoint.
- Uses your custom serializer to return extra fields like:
  - id
  - username
  - email
  - role
  - image URL
- No need for frontend to call a separate "get user info" API after login.

4. RegisterView — Create New User

```
class RegisterView(APIView):

    def post(self, request):
        serializer = RegisterSerializer(data=request.data)

        if serializer.is_valid():
            user = serializer.save()
            refresh = RefreshToken.for_user(user)
            return Response({
                "id": user.id,
                "username": user.username,
                "email": user.email,
                "role": user.role,
                "access": str(refresh.access_token),
                "refresh": str(refresh),
                "message": "User registered successfully"
            }, status=201)

        return Response(serializer.errors, status=400)
```

Medium Detailed Explanation

- Creates user with username, email, password.
- Automatically logs in user by issuing JWT tokens.
- Frontend receives everything required to store the session immediately.

5. MeView — Fetch Logged-in User Profile

```
class MeView(APIView):
    permission_classes = [UnifiedRBACPermission]

    def get(self, request):
        serializer = MeSerializer(request.user, context={'request': request})
        return Response(serializer.data)
```

Medium Detailed Explanation

- Returns the authenticated user's details.
- Uses **MeSerializer** which attaches absolute image URLs.
- Permission ensures only logged-in users can access the endpoint.

6. UpdateProfile — Edit Logged-In User

```
class UpdateProfile(APIView):
    permission_classes = [UnifiedRBACPermission]
    parser_classes = [MultiPartParser, FormParser]

    def patch(self, request):
        user = request.user
        serializer = UserUpdateSerializer(
            user,
            data=request.data,
            partial=True,
            context={"request": request}
        )

        if serializer.is_valid():
            serializer.save()
            return Response({
                "message": "Profile updated successfully",
                "user": serializer.data
            })

        return Response(serializer.errors, status=400)
```

Medium Detailed Explanation

- **PATCH** allows partial updates (only send changed fields).
- **MultiPartParser** enables image uploads such as profile pictures.
- Uses **UserUpdateSerializer** which includes email validation.
- UnifiedRBACPermission ensures:
    - User can update only their own profile
    - Admin/staff rules still apply

7. ListCreatePost — User Dashboard for Posts

```
class ListCreatePost(APIView):
    permission_classes = [UnifiedRBACPermission]

    def get(self, request):
        posts = Post.objects.filter(author=request.user).order_by("-created_at")
        serializer = PostSerializer(posts, many=True, context={"request": request})
        return Response(serializer.data)

    def post(self, request):
        serializer = CreatePostSerializer(data=request.data)

        if serializer.is_valid():
            post = serializer.save(author=request.user)
            post.save()  # triggers preview generation
            return Response(
                PostSerializer(post, context={"request": request}).data,
                status=201
            )

        return Response(serializer.errors, status=400)
```

Medium Detailed Explanation

- **GET**: fetches posts created by the logged-in user only.
- **POST**: creates a new PDF post.
- After saving, **post.save()** auto-generates preview image from PDF (using PyMuPDF).
- Response returns the full post object including absolute URLs.

8. DetailPost — Edit/Delete a Post

```
class DetailPost(APIView):
    permission_classes = [UnifiedRBACPermission]

    def patch(self, request, pk):
        post = get_object_or_404(Post, pk=pk)
        self.check_object_permissions(request, post)

        serializer = CreatePostSerializer(post, data=request.data, partial=True)

        if serializer.is_valid():
            updated = serializer.save()
            return Response(PostSerializer(updated, context={"request": request}).data)

        return Response(serializer.errors, status=400)

    def delete(self, request, pk):
        post = get_object_or_404(Post, pk=pk)
        self.check_object_permissions(request, post)
        post.delete()
        return Response({"message": "Post deleted"})
```

Medium Detailed Explanation

- **patch()**: User may edit title, description, or file.
- **delete()**: Removes the post.
- Uses **check_object_permissions()** → applies RBAC:
    - Admin can delete any post
    - Staff can delete user posts but not admin posts
    - User can delete only their own posts

9. ToggleLike — Like or Unlike a Post

```
class ToggleLike(APIView):
    permission_classes = [UnifiedRBACPermission]

    def post(self, request, post_id):
        try:
            post = Post.objects.get(id=post_id)
        except Post.DoesNotExist:
            return Response({"error": "Not found"}, 404)

        if request.user in post.likes.all():
            post.likes.remove(request.user)
            liked = False
        else:
            post.likes.add(request.user)
            liked = True

        return Response({
```

```
            "liked": liked,
            "likes_count": post.likes.count()
        })
```

Medium Detailed Explanation

- If the user already liked → remove like.
- If not → add like.
- Returns updated like count.
- RBAC ensures only valid authenticated users can toggle likes.

10. GetProfileById — Public User Profile

```
class GetProfileById(APIView):
    permission_classes = [UnifiedRBACPermission]

    def get(self, request, pk):
        user = get_object_or_404(User, id=pk)
        posts = Post.objects.filter(author=user).order_by("-created_at")

        post_serializer = HomePostSerializer(
            posts, many=True, context={"request": request}
        )

        return Response({
            "user": {
                "id": user.id,
                "username": user.username,
                "email": user.email,
                "verified": user.verified,
                "bio": getattr(user, "bio", ""),
                "image": request.build_absolute_uri(user.image.url) if user.image else None,
            },
            "posts": post_serializer.data,
        })

    def delete(self, request, pk):
        user_to_delete = get_object_or_404(User, id=pk)
        self.check_object_permissions(request, user_to_delete)
        user_to_delete.delete()
        return Response({"message": "User deleted successfully"})
```

Medium Detailed Explanation

- GET returns:
  - User details
  - All posts by that user
- DELETE allowed only if:
  - Admin attempts deletion
- Used for profile pages like `/profile/23`.

11. SinglePostView — Public Post Page

```
class SinglePostView(APIView):
    permission_classes = [UnifiedRBACPermission]

    def get(self, request, pk):
        try:
            post = Post.objects.get(pk=pk)
        except Post.DoesNotExist:
            return Response({"error": "Post not found"}, 404)

        serializer = HomePostSerializer(post, context={"request": request})
        return Response(serializer.data)
```

Medium Detailed Explanation

- Returns full post details and comments for public viewing.
- Uses **HomePostSerializer** which includes:
  - author details
  - image preview
  - likes count
  - top-level comments

12. AddCommentView — Add/Delete Comment

```
class AddCommentView(APIView):
    permission_classes = [UnifiedRBACPermission]

    def post(self, request, pk):
        text = request.data.get("text")
        if not text:
            return Response({"error": "Text is required"}, 400)

        post = get_object_or_404(Post, pk=pk)

        comment = Comment.objects.create(
            user=request.user,
            post=post,
            text=text
        )

        return Response(
            CommentMiniSerializer(comment, context={"request": request}).data,
            201
        )

    def delete(self, request, post_id, comment_id):
        comment = get_object_or_404(Comment, id=comment_id, post_id=post_id)
        self.check_object_permissions(request, comment)
        comment.delete()
        return Response({"message": "Comment deleted"})
```

Medium Detailed Explanation

- POST → Adds a new comment under the given post.
- DELETE → Removes a specific comment.
- RBAC:
  - Admin can delete any comment
  - Staff can delete user comments
  - User can delete:
    - their own comments
    - comments under their own post

13. SearchView — Search Users & Posts

```
class SearchView(APIView):

    def get(self, request):
        query = request.GET.get("q", "").strip()

        if query == "":
            return Response({"users": [], "posts": []})

        users = User.objects.filter(username__icontains=query)
        posts = Post.objects.filter(
            Q(title__icontains=query) |
            Q(description__icontains=query)
        )

        return Response({
            "users": SearchUserSerializer(users, many=True, context={"request": request}).data,
            "posts": SearchPostSerializer(posts, many=True, context={"request": request}).data,
        })
```

Medium Detailed Explanation

- **icontains** → Case-insensitive match for flexible searching.
- Search includes:
  - usernames
  - post titles
  - post descriptions
- Returns separate lists for users and posts.

14. TopUsersByLikes — User Leaderboard

```
class TopUsersByLikes(APIView):

    def get(self, request):
        users_qs = (
            User.objects
                .annotate(total_likes=Count('posts__likes'))
                .order_by('-total_likes', 'username')[:5]
        )
        serializer = TopUserSerializer(users_qs, many=True, context={"request": request})
        return Response(serializer.data)
```

Medium Detailed Explanation

- Annotates each user with the total number of likes on their posts.
- Orders users by popularity → top 5.
- Ideal for “Top Creators” section on homepage.

15. TopPostsAPIView — Most Liked Posts

```
class TopPostsAPIView(APIView):

    def get(self, request):
        posts = Post.objects.annotate(
            like_total=Count("likes")
        ).order_by("-like_total")[:5]

        serializer = TopPostSerializer(posts, many=True)
        return Response(serializer.data)
```

Medium Detailed Explanation

- Computes like count for each post.
- Sorts posts in descending order of popularity.
- Returns top 5 — used for a leaderboard or trending page.

16. PaginatedHomePosts — Paginated Feed

```
class PaginatedHomePosts(APIView):
    def get(self, request):
        page = int(request.GET.get("page", 1))
        limit = int(request.GET.get("limit", 10))

        start = (page - 1) * limit
        end = start + limit

        posts = Post.objects.all().order_by("-created_at")[start:end]
        total = Post.objects.count()

        serializer = HomePostSerializer(posts, many=True, context={"request": request})

        return Response({
            "page": page,
            "has_more": end < total,
            "results": serializer.data
        })
```

Medium Detailed Explanation

- Supports frontend infinite scroll or page-based loading.
- **page** and **limit** query params allow flexible control.
- **has_more** helps UI decide if “Load More” should appear.
- HomePostSerializer returns lightweight but complete post data.

17. ToggleVerifyView — Admin Verify/Unverify Users

```
class ToggleVerifyView(APIView):
    permission_classes = [UnifiedRBACPermission]

    def patch(self, request, pk):
        try:
            user = User.objects.get(id=pk)
        except User.DoesNotExist:
            return Response({"error": "User not found"}, 404)

        if request.user.role != "admin":
            return Response({"error": "Only admin can verify/unverify users"}, 403)

        user.verified = not user.verified
        user.save()

        return Response({
            "message": "User verified" if user.verified else "User unverified",
            "verified": user.verified
        })
```

Medium Detailed Explanation

- Only admin can toggle verification status.
- Used for moderating user accounts.
- RBAC enforces permissions through UnifiedRBACPermission + explicit role check.

urls.py

1. Imports Used in urls.py

```
from django.urls import path
from .views import (
    LoginView, RegisterView, MeView, UpdateProfile, ListCreatePost,
    DetailPost, ToggleLike, GetProfileById, SinglePostView,
    AddCommentView, SearchView, TopUsersByLikes, TopPostsAPIView,
    PaginatedHomePosts, ToggleVerifyView, GoogleAuthAPIView
)
```

Medium Detailed Explanation

- **path** — Used to define URL paths in Django.
- **views imported** — Each view corresponds to an API endpoint:
  - Authentication views (Login, Register, Google login)
  - User profile views
  - Post CRUD views
  - Comment actions
  - Search & leaderboard
  - Pagination for homepage feed
- Writing imports this way keeps the file clean, readable, and avoids long module paths.

2. URL Patterns

```
urlpatterns = [

    # Auth
    path("auth/login/", LoginView.as_view(), name="login"),
    path("auth/google/", GoogleAuthAPIView.as_view(), name="google_auth"),
    path("auth/register/", RegisterView.as_view(), name="register"),

    # Profile
    path("me/", MeView.as_view(), name="me"),
    path("profile/update/", UpdateProfile.as_view(), name="update_profile"),
    path("profile/<int:pk>/", GetProfileById.as_view(), name="profile_by_id"),
    path("profile/verify/<int:pk>/", ToggleVerifyView.as_view(), name="toggle_verify"),

    # Posts
    path("posts/", ListCreatePost.as_view(), name="list_create_posts"),
    path("posts/<int:pk>/", DetailPost.as_view(), name="post_detail"),
    path("posts/single/<int:pk>/", SinglePostView.as_view(), name="single_post"),

    # Like
    path("posts/<int:post_id>/like/", ToggleLike.as_view(), name="toggle_like"),

    # Comments
    path("posts/<int:pk>/comment/", AddCommentView.as_view(), name="add_comment"),
    path("posts/<int:post_id>/comment/<int:comment_id>/",
         AddCommentView.as_view(), name="delete_comment"),

    # Search
    path("search/", SearchView.as_view(), name="search"),

    # Leaderboards
    path("top/users/", TopUsersByLikes.as_view(), name="top_users"),
    path("top/posts/", TopPostsAPIView.as_view(), name="top_posts"),

    # Paginated Home Feed
    path("home/", PaginatedHomePosts.as_view(), name="home_posts"),
]
```

Medium Detailed Explanation

- The `urlpatterns` list maps URLs to view classes.
- Each entry uses:
  - **path()** → the URL route
  - **.as_view()** → converts class-based view into a view function
  - **name** → a unique identifier used in reverse() or for frontend clarity
- Endpoints are grouped by functionality:
  - Authentication
  - User Profile
  - Posts CRUD
  - Likes
  - Comments
  - Search
  - Leaderboards
  - Home Feed Pagination

3. Explanation of All Routes

A) Authentication Endpoints

- `auth/login/` — JWT login using username + password.
- `auth/google/` — Login/register via Google OAuth.
- `auth/register/` — Creates new user, returns JWT tokens.

B) User Profile Endpoints

- `me/` — Get authenticated user profile.
- `profile/update/` — Edit profile (bio, image, email, etc.).
- `profile/<pk>/` — Public profile with user's posts.
- `profile/verify/<pk>/` — Admin can verify/unverify user.

C) Posts Endpoints

- `posts/` — List user’s posts or create a new one.
- `posts/<pk>/` — Update or delete a post.
- `posts/single/<pk>/` — Public view of a single post.

D) Likes

- `posts/<post_id>/like/` — Like/unlike a post (toggle).

E) Comments

- `posts/<pk>/comment/` — Add a comment.
- `posts/<post_id>/comment/<comment_id>/` — Delete a comment.

F) Search

- `search/` — Search users + posts using keyword.

G) Leaderboards

- `top/users/` — Top 5 users by total likes.
- `top/posts/` — Top 5 posts by like count.

H) Home Feed

- `home/` — Paginated home feed for infinite scrolling.

📘 Settings.py

1. Imports in settings.py

```
from pathlib import Path
from datetime import timedelta
import os
```

Medium Detailed Explanation

- **Path** — Modern way to work with filesystem paths (recommended over os.path).
- **timedelta** — Used to configure JWT expiration times.
- **os** — Used for environment variables and building media/static paths.

2. Base Directory & Security Settings

```
BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'your-secret-key-here'

DEBUG = True

ALLOWED_HOSTS = ['*']
```

Medium Detailed Explanation

- **BASE_DIR** — Root directory of the Django project. Used for path joining.
- **SECRET_KEY** — Cryptographic key used for:
  - session signing
  - password reset tokens
  - CSRF protection
- **DEBUG = True** — Enables debug mode for development. On production, this MUST be set to `False`.
- **ALLOWED_HOSTS = ["*"]** — Allows requests from any domain (development-friendly).

3. Installed Apps

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Custom App
    'api',

    # Third-party
    'rest_framework',
    'corsheaders',
    'rest_framework_simplejwt',
]
```

Medium Detailed Explanation

- **Django built-in apps** provide:
  - admin dashboard
  - auth system
  - static file handling
  - session handling
- **'api'** — your custom app containing models, serializers, views, and permissions.
- **Third-party apps**:
  - **REST Framework** — enables building APIs.
  - **CORS Headers** — allows cross-origin requests from frontend (Angular, React, etc.).
  - **SimpleJWT** — adds secure JWT authentication.

4. Middleware Stack

```
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Medium Detailed Explanation

- **CorsMiddleware** — Important for SPAs (Angular, React). Allows backend API to be accessed from another domain.
- **SecurityMiddleware** — Adds security headers like HSTS and HTTPS redirects.
- **AuthenticationMiddleware** — Enables user sessions and request.user.
- **CSRF Middleware** — Protects against CSRF attacks (except JWT APIs).
- **Clickjacking Middleware** — Adds security against iframe embedding.

5. Templates Configuration

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Medium Detailed Explanation

- **APP_DIRS = True** — Enables Django to auto-detect templates inside each app.
- **DIRS** empty — no global templates folder is created.
- **context_processors** — Makes objects like `request`, `user`, and `messages` automatically available inside templates.
- This project is mainly an API, so templates are minimal.

6. Database Settings

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Medium Detailed Explanation

- Using **SQLite** — perfect for development and small projects.
- **Advantages:**
  - No setup required
  - Lightweight
  - File-based
- For production, commonly replaced with:
  - PostgreSQL
  - MySQL
  - MariaDB

7. Password Validators

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

Medium Detailed Explanation

- Ensures passwords meet security standards.
- **UserAttributeSimilarityValidator** Prevents passwords similar to username/email.
- **MinimumLengthValidator** Enforces a basic minimum length (default = 8).
- **CommonPasswordValidator** Prevents common passwords like "password123".
- **NumericPasswordValidator** Prevents passwords made only of numbers.

8. Internationalization

```
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True
```

Medium Detailed Explanation

- **LANGUAGE_CODE** — default language.
- **TIME_ZONE** — server timezone (use user's local timezone on frontend).
- **USE_I18N** — enables translation support.
- **USE_TZ** — stores datetime in UTC internally.

9. Static & Media Files

```
STATIC_URL = '/static/'

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Medium Detailed Explanation

- **STATIC_URL** — path for serving CSS, JS, and images (admin assets included).
- **MEDIA_URL** — base URL for user-uploaded files:
  - PDF files
  - Preview images
  - User profile images
- **MEDIA_ROOT** — actual folder where uploaded files are stored.

10. REST Framework + JWT + Custom API Settings

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}

CORS_ALLOW_ALL_ORIGINS = True

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(days=1),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=30),
}

GOOGLE_CLIENT_ID = 'your_google_client_id_here'

AUTH_USER_MODEL = 'api.User'
```

Medium Detailed Explanation

- **JWTAuthentication** — all protected routes use JWT instead of sessions.
- **CORS_ALLOW_ALL_ORIGINS = True** Allows your Angular frontend to call Django backend from another domain.
- **SIMPLE_JWT** config:
    - Access tokens expire in 1 day
    - Refresh tokens expire in 30 days
- **GOOGLE_CLIENT_ID** — used to verify Google ID tokens for Google OAuth login.
- **AUTH_USER_MODEL** — tells Django to use your custom user model from `api.models.User`.

📘 Django API Reference

This page provides a complete reference of all API endpoints, including their HTTP methods, authentication requirements, and purpose.

🚀 Quick API Reference

- ***/auth/login/*** – JWT Login
- ***/auth/google/*** – Google OAuth Login
- ***/auth/register/*** – Register User
- ***/me/*** – Get logged-in user
- ***/profile/update/*** – Edit profile
- ***/posts/*** – Create & list user posts
- ***/posts/<id>/*** – Edit/Delete post
- ***/posts/single/<id>/*** – Public post view
- ***/search/*** – Search posts & users
- ***/top/users/*** – Top liked creators
- ***/top/posts/*** – Trending posts
- ***/home/*** – Paginated home feed

🔐 Authentication Endpoints

Endpoint	Method	Description	Auth Required
/auth/login/	POST	Login using username and password, returns JWT tokens.	No
/auth/google/	POST	Login/Register using Google OAuth ID token.	No
/auth/register/	POST	Create a new user and return JWT tokens.	No

👤 User Profile Endpoints

Endpoint	Method	Description	Auth Required
/me/	GET	Get the logged-in user's profile.	Yes
/profile/update/	PATCH	Update user profile details.	Yes
/profile/<id>/	GET	Fetch public profile + posts of a user.	Yes (but unrestricted GET)
/profile/verify/<id>/	PATCH	Admin can verify/unverify a user.	Admin Only

📄 Post Endpoints

Endpoint	Method	Description	Auth Required
/posts/	GET	List posts created by logged-in user.	Yes
/posts/	POST	Create a new post with PDF upload.	Yes
/posts/<id>/	PATCH	Edit a post.	Yes (Owner / Staff / Admin)
/posts/<id>/	DELETE	Delete a post.	Yes (Owner / Staff / Admin)
/posts/single/<id>/	GET	Public view of a single post.	No

❤️ Like System Endpoints

Endpoint	Method	Description	Auth Required
/posts/<id>/like/	POST	Toggle like/unlike on a post.	Yes

💬 Comment Endpoints

Endpoint	Method	Description	Auth Required
/posts/<id>/comment/	POST	Add a new comment to a post.	Yes
/posts/<post_id>/comment/<comment_id>/	DELETE	Delete a specific comment.	Yes (Owner / Admin / Staff)

🔎 Search Endpoints

Endpoint	Method	Description	Auth Required
/search/	GET	Search users & posts with keyword.	No

🏆 Leaderboard Endpoints

Endpoint	Method	Description	Auth Required
/top/users/	GET	Top 5 users ranked by total likes on posts.	No
/top/posts/	GET	Top 5 most liked posts.	No

🏠 Home Feed (Pagination)

Endpoint	Method	Description	Auth Required
/home/	GET	Paginated home feed for infinite scroll.	No

📘 Complete Backend Documentation Summary

This section provides a comprehensive overview of the entire Django backend architecture, explaining how all modules work together to form a fully functional REST API for authentication, posts, comments, file uploads, search, leaderboard, and Google OAuth.

🧩 1. models.py — Data Structure & Business Logic

The **models.py** file defines the database schema and core logic for handling users, posts, comments, and file uploads. It includes:

- **User Model (Custom)** — extends Django's default user model to support roles (`admin`, `staff`, `user`), profile image, bio, verification status, and blocked status.
- **Post Model** — stores PDF posts along with an auto-generated image preview (using PyMuPDF). Handles the creation of thumbnails from uploaded PDF files.
- **Comment Model** — supports nested comments via a parent-child structure, allowing replies.
- **Upload Path Functions** — cleanly organize media files into folders: `pdfs/`, `pdf_previews/`, `userimage/`.

Together, these models provide the foundation for posts, user interactions, and file handling across your platform.

🔄 2. serializers.py — Input/Output Data Transformation

The **serializers.py** file transforms raw model instances into JSON responses and validates incoming data. It includes serializers for authentication, users, posts, comments, search, and leaderboards.

- **MyTokenObtainPairSerializer** — extends JWT login response with user details (id, role, email, profile image).
- **RegisterSerializer** — handles user account creation with secure password hashing.
- **MeSerializer & UserUpdateSerializer** — used for user profile fetch/update, supporting file uploads and dynamic absolute image URLs.
- **PostSerializer & HomePostSerializer** — provide post details, likes count, comments, preview image URL, and author info.
- **Mini Serializers** — lightweight versions used inside nested serializers (comments, trending posts, etc.).
- **Search & Top Serializers** — used for search results and leaderboard endpoints.

These serializers ensure only valid data enters the database and that all responses contain clean, structured JSON.

🔐 3. permissions.py — Unified Role-Based Access Control (RBAC)

The **UnifiedRBACPermission** class manages access rules across all models and views in a centralized system.

- **Public Read Access** — All users (even unauthenticated) can view posts and profiles.
- **Authenticated Write Access** — Only logged-in users can create posts, comment, or like.
- **Object-Level Permissions:**
    - Users can edit/delete their own posts and comments.
    - Staff can moderate user content.
    - Admins override all restrictions.
- **User Profile Protection** — Users can update only their own accounts.

This centralized permission ruleset significantly simplifies the code while ensuring strong security.

🌐 4. views.py — All API Endpoints & Application Logic

The **views.py** file contains all REST API endpoints. It is the main interaction layer between the client and the backend system.

- **Authentication:** Login (JWT), Register, and Google OAuth.
- **Profile Management:**
    - /me/ — fetch user details
    - /profile/update/ — update info + upload profile image
    - /profile/<id>/ — view public profiles
    - Admin can verify/unverify accounts
- **Posts API:** Create posts, update/delete posts, preview generation from PDF.
- **Likes:** Toggle like/unlike features with real-time count updates.
- **Comments:** Add and delete comments with a threaded structure.
- **Search API:** Searches users & posts using flexible keyword matching.
- **Leaderboard:** Top users and posts sorted by total likes.
- **Home Feed Pagination:** Infinite scroll optimized endpoint.

Views orchestrate serializers, models, and permissions to deliver a powerful and clean API interface.

🛣️ 5. urls.py — Routing & Endpoint Mapping

The **urls.py** file maps each API endpoint to its corresponding view. This makes it the navigation blueprint of your backend.

- **Authentication Routes** — login, register, Google OAuth.
- **User Routes** — profile management, verification.
- **Post Routes** — CRUD operations + public viewing.
- **Like & Comment Routes** — social interaction endpoints.
- **Search & Leaderboard Routes** — discovery and popularity ranking.
- **Home Feed Route** — paginated listings.

This file ties the entire backend together by exposing all functionality through clean, RESTful URLs.

⚙️ 6. settings.py — Core Project Configuration

The **settings.py** file defines global configuration for the project, including authentication methods, installed apps, middleware, media storage, and JWT lifetimes.

- **Installed Apps** — Django + REST Framework + CORS + SimpleJWT + custom API app.
- **REST Framework** — fully configured to use JWT authentication.
- **JWT Settings** — token lifetime, refresh token handling.
- **CORS Setup** — allows requests from any frontend domain.
- **Media Settings** — organized storage for user images, PDFs, and previews.
- **Custom User Model** — enabled via `AUTH_USER_MODEL = 'api.User'`.
- **Google OAuth Key** — integrated for seamless Google authentication.

This configuration ensures a secure, scalable, and flexible backend suitable for production-level API applications.

🔐 Role-Based Permission Table

This table summarizes the full access control rules enforced by **UnifiedRBACPermission**. It covers all actions related to users, posts, and comments.

Action	User	Staff	Admin
View Posts / Profiles (GET)	✔️ Allowed	✔️ Allowed	✔️ Allowed
Create Post	✔️ Allowed	✔️ Allowed	✔️ Allowed
Edit Own Profile	✔️ Allowed	✔️ Allowed	✔️ Allowed
Delete Post	✔️ Only own posts	✔️ Can delete user posts, cannot delete admin posts	✔️ Can delete any post
Add Comment	✔️ Allowed	✔️ Allowed	✔️ Allowed
Delete Comment	✔️ Own comment or comment on own post	✔️ Can delete user comments, not admin comments	✔️ Can delete any comment
Like / Unlike Post	✔️ Allowed	✔️ Allowed	✔️ Allowed
View Any Profile	✔️ Allowed	✔️ Allowed	✔️ Allowed
Delete User Account	✖️ Not allowed	✖️ Not allowed	✔️ Admin only
Verify / Unverify User	✖️ Not allowed	✖️ Not allowed	✔️ Admin only

📄 End of Complete Project Documentation

You now have a full overview of all modules and how they work together. This Django REST API is production-ready, scalable, and deeply integrated with modern frontend frameworks like Angular.

— QuickThrough Angular App + Django API Documentation